JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Generative AI

Generative Adversarial Networks

Outline

- Module Learning Objectives
- Generative Adversarial Networks
- Adversarial Learning
- GAN Optimization
- GAN Applications

Module Learning Objectives

- Define, analyze and apply adversarial learning by generator and discriminator networks
- Identify key applications of GANs, such as image synthesis, audio enhancement, and style transfer
- Describe the Minimax optimization framework and its role in achieving a Nash equilibrium in GAN training
- Differentiate between common GAN variants like cGAN, CycleGAN, and WGAN
- Analyze challenges in GAN training, including mode collapse and unstable optimization, and explore potential solutions
- Demonstrate GAN on Fashion MNIST dataset

Generative Adversarial Networks

- A class of machine learning frameworks developed by Ian Goodfellow in 2014
- Consist of two neural networks, a **generator** and a **discriminator**
- Yann LeCun described GANs as "the most interesting idea in machine learning in the last 10 years."
- Significant attention, with 28,500 research papers published in 2020 alone
  - Year 2023 had about 46,500 entries with "GAN" in Google Scholar
- Applications in various fields such as image synthesis, text-to-image generation, audio enhancement, and more
- Optimization goal is the **Minimax optimization** problem that seeks a **Nash equilibrium** between the generator and the discriminator

The Case of the "Lost Rembrandt"

- In 2019, researchers and curators from the Rembrandt House Museum and the Microsoft AI for Cultural Heritage Project collaborated on a revolutionary project
  - To digitally recreate a "lost" painting by Rembrandt using GANs
- In the past Rembrandt's works are lost but his distinct style carefully studied
- A GAN was "fed" not only with Rembrandt's entire catalog but also with meticulous details about his brushstrokes, use of light and shadow, and composition techniques
- The GAN played both painter and the art critic
  - replicating the method by which art forgers operated in the past
- **Reaching Beyond Human Capacity** GANs generated "false Rembrandts"
  - Some were rejected as too modern, others as too generic
- Slowly, the GAN started to capture the soul of Rembrandt's art and even **eluded master forgers**

The Ethical Debate

- Though not an "authentic" Rembrandt, the GAN-produced painting shed light on what might have been
- Museums worldwide are now exploring how these networks can restore missing parts of damaged works and even revive long-lost styles from historical artists

Adversarial Learning

- Classically, generative problems are solved by statistical methods
  - Boltzmann machines, Markov chains, variational autoencoders, etc.
- A **classification** model maps high-dimensional data to low dimensions
- A **generative** model maps low-dimensional data to high dimensions
- Combine two machines face to face
  - Output of the generative (**armor**) as input of the classification (**sword**) model
  - Performances improve training with each other
  - Both models try to weaken each other
  - As a result, improve each other and this is called **adversarial learning**
  - Models have opposite optimization goals of classification and generation
  - Models are **not** connected in a computational graph (unlike an Autoencoder)

Minimax Optimization

- Mathematical strategy often used in game theory and optimization
- Two agents or entities compete against each other
  - e.g., as players in a game or components of a model
- One trying to **maximize** a certain objective and the other trying to **minimize** it
- The minimax to find a solution that balances two competing objectives

$$\min_{x} \max_{y} f(x, y)$$

- x is controlled by one player trying to minimize the obj function $f(x, y)$
- y is controlled by the other player maximizing the same obj $f(x, y)$
- Reaches an equilibrium where neither player can improve their outcome by unilaterally changing their strategy
  - Referred to as the saddle point or **Nash equilibrium**

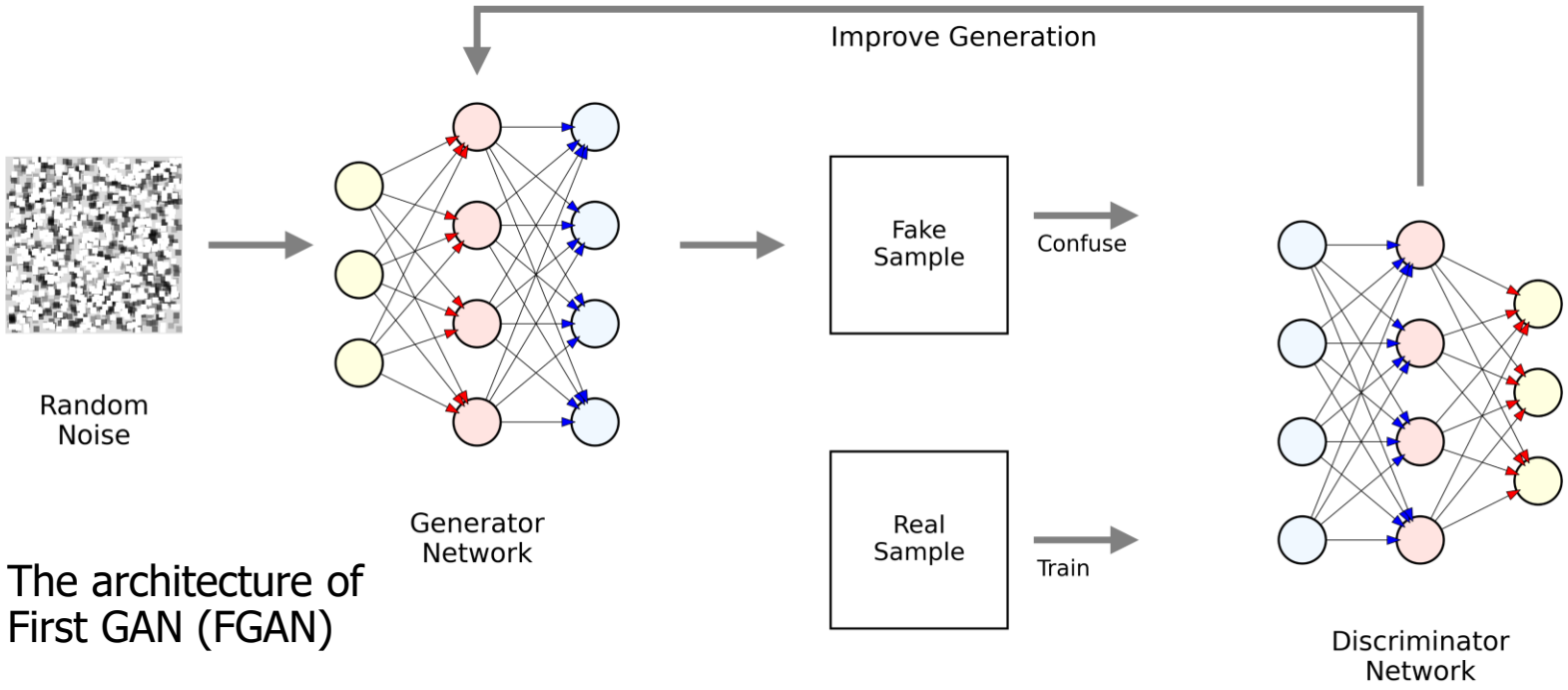
Game Theory

- Situations where there are conflicting interests among participants
- Examine mathematical models of strategic communication among logical actors
- A game can be defined by the tuple $G = (N, S, u)$
  - N as the set of participants in the game
  - $S = \{S_1, S_2, \dots, S_n\}$ the range of strategies that the players can choose from
    - Strategy space $S = S_1 \times S_2 \times \dots \times S_n$
  - Strategy profile $s = (s_1, s_2, \dots, s_n)$ specific combination chosen by the players
  - Payoff function u specifies how much utility (or payoff) each player receives
    - $u_i : S_1 \times S_2 \times \cdots \times S_n \to \mathbb{R}$
  - $u(s_1, s_2, \dots, s_n)$ indicating the utility of player i given the selection of strategies

Nash Equilibrium

- Nash Equilibrium can forecast the results in which no player benefits anymore
  - No player is motivated to change their strategy once they have taken into account the choices made by their opponents
- A strategy profile $S^* = (s_1^* s_2^* \dots s_n^*)$ is considered a Nash Equilibrium when no player can benefit from unilaterally changing their strategy
  - $u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*), \forall s_i, \forall i \in N$
  - where s_{-i}^* denotes strategic decisions made by all participants except player i
- Significance of Nash Equilibrium
  - **Predictive** provides insight what strategies rational players are likely to adopt
  - **Real-world applications** equilibria used in economics, politics, business, etc.
  - **Existence** by the Nash Equilibrium Theorem guarantees that in games with a finite number of strategies, there exists at least one Nash equilibrium

GAN Architecture



- The architecture of First GAN (FGAN)

First GAN Optimization

- Given G and D as the generator and discriminator networks, let V represent the system performance criterion
- The optimization objective is described as,

$$\min_{G} \max_{D} V(D, G) = \mathbf{E}_{x}[\log D(x)] + \mathbf{E}_{x^*}[\log (1 - D(x^*))]$$

  - where x is the real sample
  - $x^* = G(z)$ is the generated sample
  - z is the random noise that G uses to generate fake samples
  - $\mathbf{E}_{x}$ is the expectation over x
- Maximize the prediction of real samples, thus train D with gradient ascent

FGAN Optimization

- The update rule of D becomes

$$\theta_d \leftarrow \theta_d + \frac{1}{m} \nabla_{\theta_d} \sum_{i=1}^{m} (\log D(x_i) + \log(1 - D(x_i^*)))$$

  - where θ_d are all the weights in the network D
  - m is the batch size
  - i is the index of the batch
- Maximize $D(G(z))$ or minimize $1 - D(G(z))$, thus train G with gradient descent

$$\theta_g \leftarrow \theta_g + \frac{1}{m} \nabla_{\theta_g} \sum_{i=1}^{m} \log(1 - D(G(z_i))$$

  - where θ_g are all the weights in the network G

FGAN Example

- Both `Discriminator` and `Generator` are CNNs without any fully connected layers
  - They are also symmetrical in terms of network structures and sizes
  - The generator uses `ConvTranspose2d` to upsample the feature space
- Training involves three steps
  - Train the `Discriminator` with the real data and recognize it as **real**
  - Train the `Discriminator` with the fake data and recognize it as **fake**
  - Train the `Generator` with the fake data and recognize it as **real**
- Optimize `G` and `D` based on their respective losses
- During training, fake images at every epoch and network parameters are saved
  - Saved `G` network parameters (file `PTH` extension) can be used to load/generate

FGAN Example in PyTorch

```
class Generator(nn.Module):
  def __init__(self):
    super().__init__()
    self.net = nn.Sequential(
      nn.ConvTranspose2d(4*G_HIDDEN,
        2*G_HIDDEN,4,1,0,bias=False),
      nn.BatchNorm2d(2*G_HIDDEN),
      nn.ReLU(),
      nn.ConvTranspose2d(2*G_HIDDEN,
        G_HIDDEN,4,2,1,bias=False),
      nn.BatchNorm2d(G_HIDDEN),
      nn.ReLU(),
      nn.ConvTranspose2d(G_HIDDEN,
        IMG_CHANNEL,4,2,1,bias=False),
      nn.Tanh()
    )
  def forward(self,_input):
    return self.net(_input)
```

```
class Discriminator(nn.Module):
  def __init__(self):
    super().__init__()
    self.net = nn.Sequential(
      nn.Conv2d(IMG_CHANNEL,
        D_HIDDEN,4,2,1,bias=False),
      nn.LeakyReLU(0.2),
      nn.Conv2d(D_HIDDEN,
        2*D_HIDDEN,4,2,1,bias=False),
      nn.BatchNorm2d(2*D_HIDDEN),
      nn.LeakyReLU(0.2),
      nn.Conv2d(2*D_HIDDEN,
        1,4,2,0,bias=False),
      nn.Sigmoid()
    )
  def forward(self,_input):
    return self.net(
      _input).view(-1,1).squeeze(1)
```

GAN Training Challenges

- **Mode Collapse** The generator may fail to learn the full data distribution, instead producing limited variations of a few examples
  - This is a common problem in GANs
- **Unstable Training** GAN training often requires careful balancing of generator and discriminator updates
  - Optimization landscape is highly non-convex
- **Saddle Points** Convergence is often difficult because the optimization process seeks a saddle point where the gradient does not point consistently in one direction
- **Solution Approaches** Use of techniques like least-squares GAN (LSGAN), Wasserstein GAN (WGAN), and spectral normalization

GAN Variants (the Short List)

- **InfoGAN** decomposes the noise vector into incompressible noise and latent codes, discovering important features in data
- **Conditional GAN**s (cGANs) generator and discriminator are conditioned on additional information like class labels or text
- **CycleGAN** specializes in unpaired image-to-image translation using the cycle consistency loss which encourages the model to learn mappings between domains that are invertible
- **Progressive Growing GAN** (PGGAN) uses a progressively growing architecture, starting from low-resolution images and adding layers to gradually increase resolution during training that can generate high-quality images, such as realistic human faces

Applications of GANs Image Processing

- **Image Synthesis** GANs can generate realistic images, as seen in applications like image super-resolution (SRGAN, ESRGAN)
- **Image Manipulation** GANs are used for face generation, editing, and style transfer (e.g., CycleGAN for unpaired image-to-image translation)
- **Video Processing** GANs can generate and predict future frames in video sequences
- **Image Inpainting** (Image Restoration) GANs are used to fill in missing or corrupted parts of an image can restore damaged photographs or remove unwanted objects from images by content-aware image editing
- **Image Dehazing and Denoising** GANs are employed to remove noise, haze, or other visual distortions from images to improve visibility in low-light conditions or refining images captured under poor environmental conditions

Applications of GANs Sequential Data

- **Natural Language Processing** GANs generate text (e.g., IRGAN for information retrieval), dialogue, and perform text-to-image synthesis
- **Music Generation** Models like C-RNN-GAN generate music sequences
- **Speech and Audio** GANs enhance audio quality, generate realistic speech patterns
- **Time Series Data** GANs like TimeGAN generate realistic time series data (e.g., financial or patient data) which can ensure that the generated sequences preserve temporal dynamics and dependencies, allowing for accurate synthetic data in domains where real data is scarce or sensitive
- **Anomaly Detection in Sequential Data** GANs are increasingly used in detecting anomalies in sequential data such as network traffic or sensor data

Advanced GAN Applications

- **Medical Field** GANs generate synthetic medical images and assist in drug discovery
- **Data Science** GANs augment data, enhance neural network generation, and aid in privacy-preserving methods
- **Security and Adversarial Learning** GANs are used in malware detection, steganography, and adversarial machine learning
- **Style Transfer and Art Generation** Using models like CycleGAN, GANs can convert an image from one style to another, such as turning a photograph into a painting in the style of Picasso or Van Gogh
- **Finance and Fraud Detection** GANs create synthetic financial data for model testing and validation in fraud detection or risk assessment which allows financial institutions to test their models without exposing sensitive real-world data

References

- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- Goodfellow, Ian, et al. "Generative adversarial nets." Advances in neural information processing systems 27 (2014).
- Gui, Jie, et al. "A review on generative adversarial networks: Algorithms, theory, and applications." IEEE transactions on knowledge and data engineering 35.4 (2021): 3313-3332.
- The Next Rembrandt. https://news.microsoft.com/europe/features/next-rembrandt/. Accessed Sep 2024.

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.